

CTD Python Essentials

Week 5: Data Wrangling and
Aggregation



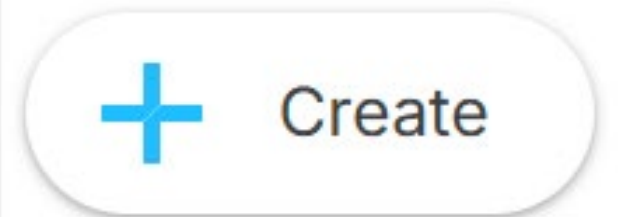
Jupyter

- Started as [Ipython](#), or Interactive Python
 - Executable documents mixing [Markdown](#), code cells, and graphics while facilitating access to high performance computing.
 - [Computational Notebook](#)
- [Jupyter](#) generalized the concepts and over time has introduced support for many languages.
- Execution options
 - Web based
 - In VSCode with plugins
 - In Kaggle
 - Many others



Kaggle

- [Kaggle](#) started in 2010 as a site which hosted data science challenges and competitions.
- Acquired by Google in 2017
- Hosts a large number of data sets
- Facilities for publishing and sharing Jupyter Notebooks
- Getting started
 - Create an account at [Kaggle: Your Home for Data Science](#)
 - Probably simplest to use your google account
 - Use the '+' in the upper left to create a new notebook with the lesson name
 - e.g. CTD_Assignment_5
 - Add cells for the tasks in the assignment
 - Leave the initial cells alone
 - Create Markdown cells to briefly describe each task
 - Level 2 headings (starting with ##) are automatically added to the table of contents
 - Follow with code cells to execute the task (triangle/arrow button at the upper left of a selected cell)
 - Variables persist between code blocks but may time out
 - You can re-run the whole notebook using 'restart and run all cells' under the three dots at the upper right of the notebook.



Data Selection

- By label
 - For single values – [at](#)
 - Potentially multiple values – [loc](#)
- By integer index
 - For single values – [iat](#)
 - Potentially multiple values – [iloc](#)
- Called using `[]`, Not `()`
- Slicing – similar to python lists
 - For labels or indices `[<start>:<end>]`
 - Omitting `<start>` or `<end>` indicates the beginning or end
- Selecting columns
 - Index the DataFrame directly `df[<col-name>]` – returns a Series
 - Multiple columns `df[[<col1>, <col2> ]]` – returns a DataFrame

```
df = pd.DataFrame({
    "col0": ['val00', 'val10', 'val20', 'val30'],
    "col1": ['val01', 'val11', 'val21', 'val31'],
    "col2": ['val02', 'val12', 'val22', 'val32'],
    "col3": ['val03', 'val13', 'val23', 'val33'],
})
df.index = ['row0', 'row1', 'row2', 'row3']
print(df)

# Column selection
print(df['col2']) # a Series
print(df[['col0', 'col3']]) # a DataFrame

# single value by label
print(df.at['row0', 'col3']) # row then column

# single value by index
print(df.iat[2,2])

# slice by label
print(df.loc['row0':'row2', 'col1':'col3'])

# slice by index
print(df.iloc[0:1, 1:3])
```

Filtering

- A column can be filtered based on the data it contains using anything which returns a Boolean value.
- A filter is a Series corresponding to a column which contains a Boolean value for each row.
 - `True` if it meets a condition, `False` if it does not
- Filters can be combined or inverted using the bitwise logical operators (
 - `&`: bitwise-and, `|`: bitwise-or, `~`: bitwise-not)
 - Don't confuse these with the Boolean logical operators (`and`, `or`, `not`)
 - Example of operator overloading
- Operator precedence is important for filters, especially when combining them
 - Best to put `()` around each filter to avoid bugs
- Several methods can be used to create filters
 - e.g. `Series.str.contains()`

```
df = pd.DataFrame({
    "Name": ['Arthur', 'Lancelot', 'Bedevere', 'Galahad'],
    "Favorite Color": ['Blue', 'Purple', 'Red', 'Green'],
    "Quest": ['Grail', 'Adventure', 'Knowledge', 'Grail'],
})
print(df)
print(df[df['Quest'] == 'Grail'])
print(df[(df['Quest'] == 'Grail') | (df['Favorite Color'] == 'Red')])
print(df[(df['Quest'] == 'Grail') & (df['Favorite Color'] == 'Blue')])
print(df[~(df['Quest'] == 'Grail')])
```

Aggregation

```
data = {
    'Category': ['A', 'A', 'B', 'B', 'C', 'C', 'C'],
    'Value1': [10, 15, 10, 20, 5, 5, 8],
    'Value2': [100, 80, 90, 100, 60, 70, 65],
}
df = pd.DataFrame(data)
print(df)
result_sum = df.groupby('Category')['Value1'].sum()
print(result_sum)
result_multiple = df.groupby('Category')['Value1'].agg(['mean', 'max'])
print(result_multiple)
multi_col_agg = df.groupby('Category')[['Value1', 'Value2']].agg(['mean', 'max'])
print(multi_col_agg)
```

- The [groupby](#) method can be used to group data by one or more columns and then apply [aggregation functions](#) to analyze or summarize the data.
 - Counts
 - Statistics
 - Custom or numpy functions
- The [agg method](#) can be used to apply multiple aggregation functions together
 - It accepts a single function, list of functions, or dict of axis label/function pairs.
- An [axis](#) in a DataFrame refers to the dimensions along which a particular method or function is applied.
 - Axis 0: rows, where axis labels are from the [index](#)
 - Axis 1: columns, where axis labels are from [column names](#)
 - For column-wise summaries, aggregation is on axis 0

	Category	Value1	Value2
0	A	10	100
1	A	15	80
2	B	10	90
3	B	20	100
4	C	5	60
5	C	5	70
6	C	8	65

Category	
A	25
B	30
C	18

	Name: Value1, dtype: int64	
	mean	max
Category		
A	12.5	15
B	15.0	20
C	6.0	8

	Value1		Value2	
	mean	max	mean	max
Category				
A	12.5	15	90.0	100
B	15.0	20	95.0	100
C	6.0	8	65.0	70

Merge

- Using the `pd.merge()` method
 - Useful for shared keys
 - Specify which columns in both DataFrames should match up using (`on`, or `left_on` / `right_on`)
 - `how` parameter
 - `'inner'` (default): only keys present in both tables are included
 - `'left'`: all keys from left (the first DataFrame), and matching keys from right
 - `'right'`: all keys from right (the second DataFrame), and matching keys from left
 - `'outer'`: all keys from both DataFrames, filling missing values with NaN where keys don't match

```
sales_data = {
    'customer_id': [1, 2, 3, 4],
    'amount': [100, 200, 150, 300]
}
df_sales = pd.DataFrame(sales_data)
print('df sales:\n', df_sales)
customer_data = {
    'customer_id': [1, 2, 5],
    'name': ['Alice', 'Bob', 'Eve']
}
df_customers = pd.DataFrame(customer_data)
print('df customers:\n', df_customers)
# Only includes rows where the customer_id exists in both DataFrames
merged_inner = pd.merge(df_sales, df_customers, on='customer_id', how='inner')
print('inner:\n', merged_inner)
# Keeps all rows from df_sales (the left DataFrame)
merged_left = pd.merge(df_sales, df_customers, on='customer_id', how='left')
print('left:\n', merged_left)
# Keeps all rows from df_customers (the right DataFrame)
merged_right = pd.merge(df_sales, df_customers, on='customer_id', how='right')
print('right:\n', merged_right)
# Keeps all rows from both DataFrames. Fills with NaN where there is no match
merged_outer = pd.merge(df_sales, df_customers, on='customer_id', how='outer')
print('outer:\n', merged_outer)
```

```
df sales:
  customer_id  amount
0           1     100
1           2     200
2           3     150
3           4     300
df customers:
  customer_id  name
0           1  Alice
1           2   Bob
2           5   Eve
inner:
  customer_id  amount  name
0           1     100  Alice
1           2     200   Bob
left:
  customer_id  amount  name
0           1     100  Alice
1           2     200   Bob
2           3     150   NaN
3           4     300   NaN
right:
  customer_id  amount  name
0           1    100.0  Alice
1           2    200.0   Bob
2           5         NaN  Eve
outer:
  customer_id  amount  name
0           1    100.0  Alice
1           2    200.0   Bob
2           3    150.0   NaN
3           4    300.0   NaN
4           5         NaN  Eve
```

Join

- Using the `DataFrame.join()` method
 - Index based
 - Typically used as a method on one `DataFrame`:
`df1.join(df2, ...)`
 - By default, join uses the `index` of both `DataFrames` as the key and does a left join (`how='left'`)
 - Optionally, you can specify a column (in the other `DataFrame`) to join on using the `on` parameter (but only if the calling `DataFrame` has its `index` as the key)
 - Useful for quickly joining on indices or combining multiple `DataFrames` that share the same or similar `index`

```
data_left = {
    'A': [10, 20, 30],
    'B': [40, 50, 60]
}
df_left = pd.DataFrame(data_left, index=['x', 'y', 'z'])
print("df_left:\n", df_left)
data_right = {
    'C': [7, 8, 9],
    'D': [1, 2, 3]
}
df_right = pd.DataFrame(data_right, index=['x', 'y', 'w'])
print("\ndf_right:\n", df_right)
# Rows from df_left (x, y, z) are preserved.
join_default = df_left.join(df_right)
print('\njoin_default:\n', join_default)
# include all indices (x, y, z, w) and fill missing values with NaN
join_outer = df_left.join(df_right, how='outer')
print('\njoin_outer:\n', join_outer)
```

```
df_left:
   A  B
x  10 40
y  20 50
z  30 60

df_right:
   C  D
x   7   1
y   8   2
w   9   3

join_default:
   A  B   C  D
x  10 40  7.0 1.0
y  20 50  8.0 2.0
z  30 60  NaN NaN

join_outer:
   A  B   C  D
w  NaN NaN  9.0 3.0
x  10.0 40.0 7.0 1.0
y  20.0 50.0 8.0 2.0
z  30.0 60.0 NaN NaN
```

Data Transformation

- You can use `[]` on a `DataFrame` to access a `View` of a `column` as a `Series`
- You can also assign a `Series` to such a `View`
- Various operations can be performed on the values in a `Series`
 - Elementwise `+`, `-`, `*`, `/`, `&`, `|`, and `~` (numbers or as overloaded for the value)
 - String operations
 - Conditionals (makes a filter)
 - `numpy` functions
 - `map` can be used to apply any function
- Various utilities
 - `df.rename(columns={'Score':'Test Score'}, inplace=True)`
 - `df.set_index('Name')` # column becomes row index
 - `df.sort_values(by='Level', ascending=False, inplace=True)`
 - `df.reset_index(inplace=True, drop=True)` # after sorting, merging, etc. munges index
 - `drop` removes the original `index` rather than making it a `column`

Q&A and Demo

This week's examples:
[CTD Python Essentials Week 5 Examples](#)

